

Librerías Python para Ciberseguridad Defensiva: hashlib, yara-python e impacket

Monografía de Investigación

Curso: Programación Aplicada a la Ciberseguridad

Alvaro Huacac Soto

Instituto de Educación Superior Tecnológico

29 de junio de 2026

Repositorio del proyecto:

<https://github.com/AlvaroHuacacSoto/hashlib-Jara>

1. Introducción

Python se ha consolidado como uno de los lenguajes de programación más relevantes en el campo de la seguridad informática. Su sintaxis clara, la disponibilidad de miles de librerías especializadas y su extensa comunidad lo convierten en la herramienta de elección para analistas, investigadores y profesionales de la ciberseguridad defensiva en todo el mundo.

El presente trabajo de investigación monográfico examina tres librerías fundamentales del ecosistema Python orientadas a la ciberseguridad: `yara-python`, `hashlib` e `impacket`. Estas herramientas permiten, respectivamente, detectar y clasificar malware mediante reglas de reconocimiento de patrones, verificar la integridad de archivos y mensajes mediante funciones hash criptográficas, y analizar protocolos de red como SMB, Kerberos y LDAP en entornos de laboratorio.

Como componente práctico principal, se desarrolló un caso real de análisis forense sobre tres aplicaciones Android (APK): una original `CairosOriginal.apk`, otra infectada con un payload de Metasploit `CairosVenom.apk`, y un APK standalone `payload.apk` generado por `MsfVenom`. Se utilizó `hashlib` para calcular y comparar los hashes criptográficos de los archivos, evidenciando el efecto avalancha, y se implementó un detector basado en `yara-python` y `Androguard` capaz de identificar automáticamente el APK envenenado mediante el análisis de permisos, subpaquetes inyectados y firmas DEX. Todo el código fuente, los APK de prueba y este documento se encuentran disponibles en el repositorio del proyecto.

1.1. Propósito de la Guía

- Orientar la elaboración del trabajo monográfico sobre librerías de Python aplicadas a la ciberseguridad.
- Relacionar la investigación teórica con una práctica funcional ejecutada con archivos APK reales.
- Promover el análisis de casos reales y la interpretación de resultados en contextos de seguridad informática.
- Demostrar el valor defensivo de `yara-python`, `hashlib` e `impacket` en entornos académicos y profesionales.

2. Planteamiento General del Tema

2.1. Descripción del Tema

Las librerías de Python son componentes de software precompilados que ofrecen funcionalidades específicas listas para ser integradas en proyectos de programación. En el ámbito de la ciberseguridad, estas librerías reducen drásticamente el tiempo necesario para implementar mecanismos defensivos, ya que los analistas pueden enfocarse en la lógica de detección y respuesta

en lugar de construir desde cero cada función criptográfica o de análisis de protocolos.

El problema que este trabajo aborda es la necesidad de contar con herramientas de detección de amenazas, verificación de integridad y análisis de tráfico de red que sean accesibles, documentadas y aplicables en entornos educativos. Las tres librerías seleccionadas (yara-python, hashlib e impacket) responden directamente a esta necesidad: yara-python permite identificar malware mediante firmas de patrones; hashlib garantiza la integridad de archivos mediante funciones hash como SHA-256 y MD5; e impacket facilita el análisis y la simulación de protocolos de red como SMB y Kerberos en laboratorio.

2.2. Importancia del Uso de Librerías en Python

Las librerías permiten reutilizar código ampliamente probado por la comunidad, reducir errores humanos en implementaciones críticas de seguridad y acceder a algoritmos complejos con interfaces de programación simples. En ciberseguridad, donde un error de implementación puede comprometer la totalidad de un sistema, el uso de librerías estandarizadas resulta indispensable.

Además, las librerías facilitan el trabajo colaborativo entre equipos de seguridad, la integración con plataformas de análisis de amenazas y la automatización de tareas repetitivas como el cálculo masivo de hashes o el escaneo de archivos con reglas YARA.

2.3. Área de Aplicación Seleccionada

Este trabajo se ubica en el área de ciberseguridad defensiva. Las librerías seleccionadas contribuyen de forma complementaria a la protección de sistemas: hashlib para integridad, yara-python para detección de código malicioso e impacket para análisis de protocolos de red en entornos controlados.

Tabla 1: Matriz de preguntas guía del proyecto

Pregunta guía	Respuesta desarrollada
¿Qué área eligieron?	Ciberseguridad defensiva: yara-python, hashlib e impacket.
¿Por qué esa área?	La ciberseguridad es crítica en la formación tecnológica actual. Las amenazas digitales crecen y las organizaciones requieren profesionales en herramientas defensivas.
¿Qué problema resuelve?	Detección de malware en APKs, verificación de integridad de software distribuido, y análisis de tráfico SMB/Kerberos en auditorías.
¿Qué resultados esperan?	Hashes verificables, detección de patrones maliciosos en APKs reales, y documentación de herramientas de análisis de red.

3. Objetivos del Trabajo

3.1. Objetivo General

Analizar el uso, características y aplicación práctica de las librerías yara-python, hashlib e impacket de Python orientadas a la ciberseguridad defensiva, mediante una investigación monográfica y el desarrollo de scripts funcionales de análisis forense de APKs.

3.2. Objetivos Específicos

1. Identificar las principales características, funciones y métodos de las librerías yara-python, hashlib e impacket.
2. Explicar el funcionamiento básico, la instalación y la importación de cada librería, junto con sus métodos principales.
3. Comparar las tres librerías según su uso principal, nivel de dificultad, ventajas, limitaciones y campo de aplicación real.
4. Desarrollar scripts en Python utilizando las librerías para resolver un problema real de ciberseguridad defensiva (detección de APKs envenenados).
5. Interpretar los resultados obtenidos y relacionarlos con un caso práctico en contexto de seguridad informática.

3.3. Coherencia de Objetivos

Tabla 2: Verificación de coherencia entre elementos del trabajo

Elemento	Verificación
Título	Menciona las tres librerías de Python en ciberseguridad.
Objetivo general	Coincide con la finalidad: análisis e implementación práctica.
Objetivos específicos	Organizan el desarrollo de lo teórico a lo práctico.
Parte práctica	Evidencia el cumplimiento mediante código ejecutado sobre APKs reales.

4. Marco Teórico

4.1. Concepto de Python

Python es un lenguaje de programación de alto nivel, interpretado, de tipado dinámico y de propósito general, creado por Guido van Rossum y publicado por primera vez en 1991 (Van Rossum & Drake, 2009). Se caracteriza por su sintaxis legible que prioriza la claridad del código, lo que lo convierte en uno de los lenguajes más utilizados tanto en educación como en entornos profesionales. En la actualidad, Python lidera índices de popularidad como el TIOBE Index y es la primera opción en campos como la ciencia de datos, la inteligencia artificial, la automatización y la ciberseguridad.

4.2. Concepto de Librería en Programación

Una librería es un conjunto de módulos, clases, funciones y recursos precompilados que permiten resolver tareas específicas dentro de un programa sin necesidad de implementar cada funcionalidad desde cero. Las librerías encapsulan lógica compleja y son distribuidas de forma que cualquier desarrollador pueda integrarlas mediante una instrucción de importación. A continuación se muestra un ejemplo de importación:

Listing 1: Importación de la librería estándar hashlib en Python

```
1 import hashlib
2 # hashlib permite calcular funciones hash criptograficas
3 # como SHA-256, MD5 y SHA-512 sin instalacion adicional
```

En este ejemplo, la instrucción `import hashlib` carga el módulo completo en el espacio de nombres del programa. A partir de ese momento, todas las funciones y clases de la librería están disponibles mediante la notación de punto (ej. `hashlib.sha256()`).

4.3. Clasificación de Librerías de Python

Tabla 3: Clasificación de librerías Python por área de aplicación

Clasificación	Descripción	Ejemplos
Analítica de datos	Limpiar, transformar y visualizar datos estructurados.	Pandas, NumPy, Matplotlib
IA y ML	Modelos que aprenden de datos y realizan predicciones.	TensorFlow, PyTorch, Scikit-learn
Procesamiento de lenguaje	Análisis de textos, palabras y lenguaje humano.	NLTK, spaCy, Transformers
Ciberseguridad defensiva	Cifrar, verificar integridad, analizar archivos y red.	hashlib, yara-python, impacket, Scapy

4.4. Importancia de las Librerías en Problemas Reales

En ciberseguridad, las librerías Python tienen un impacto directo en la capacidad de respuesta ante amenazas. `hashlib` permite verificar si un archivo ha sido alterado comparando su hash con el valor de referencia; `yara-python` permite escanear archivos en busca de firmas de malware conocidas; `impacket` es utilizada en auditorías para simular ataques sobre protocolos y detectar vulnerabilidades antes de su explotación.

5. Desarrollo de las Librerías Seleccionadas

5.1. Librería N.º 1: hashlib

5.1.1. Nombre de la Librería

Nombre oficial: `hashlib`. Se importa con `import hashlib`. Es parte de la biblioteca estándar de Python desde la versión 2.5, sin requerir instalación adicional (Python Software Foundation, 2024).

5.1.2. Definición

`hashlib` proporciona interfaces para algoritmos de hash criptográficos: MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512 y BLAKE2. Una función hash transforma datos de entrada

en una cadena de longitud fija (digest), de manera que un cambio mínimo produce un resumen completamente diferente: el efecto avalancha (Python Software Foundation, 2024).

5.1.3. Características Principales

- Algoritmos MD5, SHA-1, SHA-256, SHA-512 y BLAKE2 sin instalación.
- Cálculo de hashes de texto, archivos y flujos de datos.
- Compatible con Windows, Linux y macOS.
- Nivel de dificultad: básico.
- Integrable con hmac y secrets para aplicaciones criptográficas.

5.1.4. Instalación e Importación

hashlib está incluida en la biblioteca estándar. No requiere `pip install`:

Listing 2: Carga de hashlib sin instalación

```
1 # No se necesita instalacion - hashlib es parte de la stdlib
2 import hashlib
3 print(hashlib.algorithms_available)
```

5.1.5. Funciones Principales

Tabla 4: Métodos principales de la librería hashlib

Función	Propósito	Ejemplo
<code>hashlib.sha256()</code>	Crea objeto hash SHA-256.	<code>h = hashlib.sha256()</code>
<code>hashlib.md5()</code>	Crea objeto hash MD5 (obsoleto).	<code>h = hashlib.md5()</code>
<code>h.update(data)</code>	Alimenta datos al hash.	<code>h.update(b"texto")</code>
<code>h.hexdigest()</code>	Retorna hash hexadecimal.	<code>print(h.hexdigest())</code>
<code>h.digest()</code>	Retorna hash como bytes.	<code>h.digest()</code>
<code>hashlib.new("sha512", data)</code>	Crea hash por nombre de algoritmo.	<code>hashlib.new("sha512", b"dato")</code>

5.1.6. Ejemplo Práctico con APK Real

El siguiente fragmento, extraído del script `comparar_hashes.py` publicado en el repositorio del proyecto, calcula simultáneamente MD5, SHA-1, SHA-256 y SHA-512 de dos APK de 52 MB cada uno, leyendo el archivo en bloques de 4096 bytes para no saturar la memoria RAM:

Listing 3: Cálculo simultáneo de cuatro hashes sobre APK de 52 MB

```
1 import hashlib, os, time
2
3 DIR = "/home/alim/exposicion final/APKs"
4 ARCHIVOS = ["CairosOriginal.apk", "CairosVenom.apk"]
5
6 def calcular_hashes(ruta):
7     """Calcula MD5, SHA-1, SHA-256 y SHA-512 en una sola lectura."""
8     algoritmos = {
9         "MD5": hashlib.md5(),
10        "SHA-1": hashlib.sha1(),
11        "SHA-256": hashlib.sha256(),
12        "SHA-512": hashlib.sha512(),
13    }
14    with open(ruta, "rb") as f:
15        # Lectura por bloques de 4 KB para archivos grandes
16        for chunk in iter(lambda: f.read(4096), b''):
17            for h in algoritmos.values():
18                h.update(chunk)
19    return {nombre: obj.hexdigest()
20            for nombre, obj in algoritmos.items()}
```

Explicación del código. La función `calcular_hashes` recibe la ruta de un archivo y construye un diccionario con cuatro objetos hash inicializados. El bucle `for chunk in iter(...)` lee el archivo en fragmentos de 4 KB; en cada iteración, los cuatro objetos hash son actualizados con `update(chunk)`. Al finalizar, se retorna un diccionario con los cuatro digests en formato hexadecimal. Este diseño evita cargar el archivo completo de 52 MB en memoria y reduce el tiempo de ejecución a 0.37 segundos al calcular los cuatro algoritmos en una sola pasada (Huacac Soto, 2026).

Demostración del efecto avalancha. Una vez obtenidos los hashes, el script aplica una operación XOR bit a bit entre los bytes de los dos SHA-256 para cuantificar cuántos bits cambiaron:

Listing 4: Cálculo del efecto avalancha con XOR sobre SHA-256

```
1 # Comparacion bit a bit de los digests SHA-256
2 h1 = bytes.fromhex(original["SHA-256"])
3 h2 = bytes.fromhex(envenenado["SHA-256"])
4 bits_distintos = sum(bin(a ^ b).count('1')
5                       for a, b in zip(h1, h2))
6 total_bits = len(h1) * 8
7 porcentaje = (bits_distintos / total_bits) * 100
8 # Resultado: 130/256 bits (50.8%)
```

El resultado obtenido fue 130 bits diferentes de 256 (50.8 %). Esto confirma que la inyección del payload de Metasploit —que solo agrega 30,679 bytes al archivo— alteró radicalmente el hash, haciendo imposible predecir el nuevo valor a partir del original. Este es el principio

fundamental de la verificación de integridad criptográfica.

5.1.7. Aplicación en Contexto Real

Se aplicó hashlib al análisis forense de dos APK reales: `CairosOriginal.apk` (52,400,958 bytes) y `CairosVenom.apk` (52,431,637 bytes). El segundo fue generado mediante `MsfVenom` inyectando un payload `meterpreter_reverse_tcp`. El script `comparar_hashes.py` calculó los cuatro hashes, obteniendo valores completamente diferentes en los cuatro algoritmos, como se muestra en la Tabla 5.

Tabla 5: Comparación de hashes entre APK original y envenenado

Algoritmo	CairosOriginal.apk	CairosVenom.apk
MD5	3392a4d54f89fc7fe9c6618ff418bd75	8bd7520a63573b0225c49be3a3f55295
SHA-1	1e4dc3d80c74e...	cfb3bba3068982c7...
SHA-256	6752e6831ed26...	fe1452b4900c92...
SHA-512	201a5bacda494...	3708cf576beda8...

Nota. Los cuatro algoritmos producen valores completamente diferentes. Solo la inyección de 30 KB de payload fue suficiente para cambiar radicalmente todos los hashes. Adaptado de «hashlib-Jara» por Huacac Soto (2026).

5.1.8. Ventajas y Limitaciones

Tabla 6: Ventajas y limitaciones de hashlib

Ventajas	Limitaciones
Incluida en Python sin instalación.	MD5 y SHA-1 inseguros para uso criptográfico.
Interfaz simple y documentación oficial extensa.	No cifra datos, solo genera resúmenes.
Múltiples algoritmos en una sola API.	Para contraseñas usar <code>bcrypt</code> o <code>argon2</code> .
Eficiente con archivos grandes (lectura por bloques).	No gestiona claves ni certificados digitales.

5.2. Librería N.º 2: yara-python

5.2.1. Nombre de la Librería

Nombre oficial: `yara-python`. Se instala con `pip install yara-python` y se importa como `import yara`. Es la interfaz Python del motor YARA, la herramienta de detección de malware basada en reglas más utilizada en análisis forense digital (VirusTotal/Google, 2024).

5.2.2. Definición

yara-python expone la API de YARA, un motor de detección de patrones diseñado para identificar y clasificar malware mediante reglas que combinan cadenas de texto, patrones hexadecimales y condiciones lógicas. Fue creado por Víctor Manuel Álvarez (2014) y es mantenido por VirusTotal (Google).

5.2.3. Características Principales

- Reglas de detección con texto, bytes hexadecimales y expresiones regulares.
- Análisis de archivos, directorios y procesos en memoria.
- Nivel de dificultad: intermedio (requiere sintaxis YARA).
- Base de plataformas como VirusTotal, Cuckoo Sandbox y MISP.
- Más de 1000 reglas públicas disponibles en repositorios comunitarios.

5.2.4. Instalación e Importación

Listing 5: Instalación e importación de yara-python

```
1 # Instalacion
2 pip install yara-python
3
4 # Importacion
5 import yara
```

5.2.5. Funciones Principales

Tabla 7: Métodos principales de yara-python

Función	Propósito	Ejemplo
<code>yara.compile(source=s)</code>	Compila reglas desde cadena.	<code>r = yara.compile(source=s)</code>
<code>yara.compile(filepath=f)</code>	Compila desde archivo .yar.	<code>r = yara.compile(filepath=f"archivo.yar")</code>
<code>regla.match(filepath=f)</code>	Escanea archivo y retorna matches.	<code>matches = r.match(filepath="f.exe")</code>
<code>regla.match(data=)</code>	Escanea datos en memoria.	<code>r.match(data=contenido)</code>
<code>matches[i].rule</code>	Nombre de la regla activada.	<code>print(matches[0].rule)</code>
<code>matches[i].strings</code>	Cadenas que activaron la regla.	<code>print(matches[0].strings)</code>

5.2.6. Ejemplo Práctico: Detección de Payload en DEX

El siguiente fragmento del script `detector_yara.py` define dos reglas YARA para detectar payloads de Metasploit dentro del archivo `classes.dex` de un APK Android. Las reglas se compilan en tiempo de ejecución y se aplican sobre el DEX extraído:

Listing 6: Reglas YARA para detección de payload Metasploit en DEX de APK

```
1 import yara
2
3 REGLAS_DEX = yara.compile(source="""
4     rule dex_metasploit_payload {
5         meta:
6             description = "Detecta payload de Metasploit en DEX"
7             severity = "critical"
8         strings:
9             $a = "Ljava/lang/reflect/Method;"
10            $b = "Ljava/net/URLConnection;"
11            $c = "addRequestProperty"
12            $d = ";->startService(Landroid/content/Context;)V"
13            $e = ";->openConnection()Ljava/net/URLConnection;"
14        condition:
15            uint32(0) == 0x0A786564 and
16            4 of them
17    }
18
19    rule dex_payload_backdoor {
20        meta:
21            description = "Estructura típica de RAT/backdoor Android"
22            severity = "high"
23        strings:
24            $ref = "Ljava/lang/reflect/Method;"
25            $start = ";->startService(Landroid/content/Context;)V"
26        condition:
27            uint32(0) == 0x0A786564 and
28            all of them
29    }
30 """)
```

Explicación de las reglas. La regla `dex_metasploit_payload` busca cinco indicadores característicos de un RAT Android: uso de reflexión (`java/lang/reflect/Method`) para invocar métodos dinámicamente, conexiones de red (`java/net/URLConnection`) para comunicarse con el servidor de comando y control, envío de cabeceras HTTP (`addRequestProperty`), inicio de servicios remotos (`; ->startService`) y apertura de conexiones URL (`; ->openConnection()`). La condición exige al menos cuatro de estos cinco patrones y verifica que el archivo comience con el magic number DEX (`0x0A786564`), evitando falsos positivos en archivos no ejecutables.

(Huacac Soto, 2026).

Análisis combinado con Androguard. El detector complementa YARA con heurísticas del manifiesto Android mediante la librería Androguard:

Listing 7: Análisis heurístico del AndroidManifest con Androguard

```
1 from androguard.core.apk import APK
2
3 PERMISOS_PELIGROSOS = [
4     "android.permission.READ_SMS",
5     "android.permission.CAMERA",
6     "android.permission.RECORD_AUDIO",
7     "android.permission.READ_CALL_LOG",
8     # ... 10 permisos mas monitoreados
9 ]
10
11 apk = APK(ruta_apk)
12 permisos = apk.get_permissions()
13 # Contar cuantos permisos peligrosos se declararon
14 peligrosos = [p for p in permisos
15               if p in PERMISOS_PELIGROSOS]
16
17 # Detectar subpaquetes inyectados con nombres aleatorios
18 main_package = apk.get_package()
19 subpaquetes = [p for p in paquetes
20                if p.startswith(main_package)
21                and p != main_package]
```

El código anterior extrae la lista de permisos del AndroidManifest.xml mediante Androguard, filtra aquellos clasificados como peligrosos y detecta subpaquetes inyectados con nombres aleatorios de 5 caracteres (generados por MsfVenom al embeber el payload). En el APK envenenado se encontraron 14 permisos peligrosos y un subpaquete `com.example.cairos.dfwto` (Huacac Soto, 2026).

5.2.7. Aplicación en Contexto Real

El detector analizó tres APK y clasificó correctamente el original como LIMPIO y los dos envenenados como ENVENENADOS, como se resume en la Tabla 8.

Tabla 8: Resultados del detector YARA + Androguard sobre los tres APK

Indicador	CairosOriginal	CairosVenom	payload.apk
Tamaño	52.4 MB	52.4 MB	10 KB
Permisos totales	2	24	23
Permisos peligrosos	0	14	14
Subpaquetes extras	0	1 (dfwto)	0
Tamaño DEX	812 KB	993 KB	20 KB
Veredicto	LIMPIO	ENVENENADO	ENVENENADO

Nota. El sistema de puntuación combina permisos peligrosos, subpaquetes inyectados y tamaño de DEX. Puntuación ≥ 5 = ENVENENADO. Adaptado de «hashlib-Jara» por Huacac Soto (2026).

5.2.8. Ventajas y Limitaciones

Tabla 9: Ventajas y limitaciones de yara-python

Ventajas	Limitaciones
Estándar industrial para detección de malware.	Requiere sintaxis YARA para reglas efectivas.
Combina strings, bytes, regex y lógica booleana.	Reglas mal escritas generan falsos positivos.
Miles de reglas públicas en GitHub y VirusTotal.	No detecta malware polimórfico avanzado.
Escanea archivos, procesos y datos en memoria.	Necesita actualización constante de reglas.

5.3. Librería N.º 3: impacket

5.3.1. Nombre de la Librería

Nombre oficial: impacket. Instalación: `pip install impacket`. Su importación es modular: `from impacket.smbconnection import SMBConnection,from impacket import smb, ntlm`.

5.3.2. Definición

impacket es una colección de clases Python para protocolos de red de bajo nivel en entornos Windows: SMB1/2/3, MSRPC, NTLM, Kerberos, LDAP, MSSQL, WMI y DCOM (Fortra, 2024). Es mantenida por la comunidad y base de herramientas como secretsdump, wmiexec y crackmapexec.

5.3.3. Características Principales

- Implementa SMB, Kerberos, NTLM, LDAP, MSSQL y WMI en Python puro.
- Construcción y análisis de paquetes a nivel de protocolo.
- Nivel avanzado: requiere conocimientos de redes Windows.
- Más de 20 scripts de ejemplo listos para auditorías.
- Base de BloodHound, crackmapexec y secretsdump.

5.3.4. Instalación e Importación

Listing 8: Instalación e importación modular de impacket

```

1 # Instalacion
2 pip install impacket
3
4 # Importacion por modulo
5 from impacket.smbconnection import SMBConnection
6 from impacket import smb, ntlm
7 from impacket.krb5 import kerberosv5

```

5.3.5. Funciones Principales

Tabla 10: Métodos principales de impacket

Módulo	Propósito	Ejemplo
SMBConnection()	Conexión SMB a servidor.	conn = SMBConnection(ip, ip)
conn.login(u, p)	Autenticación con credenciales.	conn.login(user", "pass")
conn.listShares()	Lista recursos compartidos.	shares = conn.listShares()
ntlm.getNTLMSSPType1()	Mensaje NTLM Tipo 1.	msg = getNTLMSSPType1()
kerberosv5.getKerberosTGT()	Ticket TGT Kerberos.	tgt = getKerberosTGT(...)

5.3.6. Limitaciones para la Documentación Práctica

Durante el desarrollo de esta monografía se encontraron barreras técnicas y legales que impidieron la ejecución de pruebas reales con impacket:

1. **Dependencia de Active Directory real.** Los módulos Kerberos requieren un Key Distribution Center (KDC) funcional. Sin un controlador de dominio Windows Server configurado con usuarios y políticas de autenticación, las funciones `getKerberosTGT()` y `getKerberosTGS()` retornan errores de conexión irrecuperables. Montar un laboratorio completo con Windows Server excede el alcance de este trabajo.
2. **Necesidad de credenciales privilegiadas.** Scripts como `secretsdump.py` y `GetNPUsers.py` requieren cuentas de dominio con privilegios específicos (ej. permisos de replicación para DCSync). La generación de estas credenciales implica administración completa de un dominio, requiriendo autorización institucional formal.
3. **Complejidad del código base.** `impacket` implementa más de 20 módulos con estructuras ASN.1, mecanismos GSS-API, cifrado RC4-HMAC y AES para Kerberos, y parsing de PAC (Privilege Attribute Certificate). Documentar un flujo completo requeriría analizar miles de líneas de código que operan en capas OSI profundas.
4. **Implicaciones éticas y legales.** Ejecutar herramientas de `impacket` contra sistemas sin autorización constituiría un delito informático según la Ley N.º 30096 (Ley de Delitos Informáticos del Perú). Por ello, la documentación se limita al análisis conceptual.

No obstante, `impacket` es una herramienta de valor incalculable en auditorías profesionales. Su inclusión responde al objetivo de documentar su arquitectura y aplicaciones teóricas (Fortra, 2024).

5.3.7. Aplicación en Contexto Real

Un auditor de seguridad utiliza `impacket` en entorno controlado para verificar configuraciones SMB expuestas y vulnerabilidades NTLM en servidores Windows, con autorización formal de la organización.

5.3.8. Ventajas y Limitaciones

Tabla 11: Ventajas y limitaciones de `impacket`

Ventajas	Limitaciones
Protocolos Windows en Python puro.	Nivel avanzado: requiere redes Windows.
Base de herramientas profesionales.	Uso no autorizado: delito informático.
Análisis a nivel de paquete de protocolos.	Documentación técnica densa.
Actualización constante de protocolos.	Requiere laboratorio con VMs Windows.

6. Cuadro Comparativo de las Librerías

Tabla 12: Cuadro comparativo de hashlib, yara-python e impacket

Criterio	hashlib	yara-python	impacket
Área	Integridad criptográfica.	Detección de malware.	Análisis de protocolos.
Uso principal	Calcular hashes SHA-256, MD5.	Escanear con reglas YARA.	Implementar SMB, Kerberos.
Nivel	Básico.	Intermedio.	Avanzado.
Datos	Archivos, strings, bytes.	Archivos, memoria, flujos.	Paquetes de red, tickets.
Instalación	Incluida (stdlib).	<code>pip install yara-python</code> .	<code>pip install impacket</code> .
Ventaja clave	Sin instalación, API simple.	Estándar industrial antimalware.	Protocolos reales en Python.
Limitación	MD5/SHA-1 obsoletos.	Requiere sintaxis YARA.	Necesita dominio y credenciales.
Práctica	Hashes de APK 52 MB en 0.37s.	Detección de payload en DEX + Androguard.	Documentación teórica.

Nota. La práctica con impacket fue documentada teóricamente por las barreras de autenticación descritas en la Sección 4.3.6.

6.1. Análisis Interpretativo

hashlib resultó la más accesible por su inclusión en la biblioteca estándar, procesando APK de 52 MB en 0.37 segundos con cuatro hashes simultáneos. La comparación XOR del SHA-256 cuantificó el efecto avalancha en 50.8 %. yara-python, con nivel intermedio, demostró que la combinación de reglas sobre el DEX con heurísticas de Androguard produce clasificación precisa de APKs envenenados. impacket, de nivel avanzado, no pudo ser probada en la práctica por requerir un KDC funcional y credenciales de dominio, pero se documentó su arquitectura.

Las tres librerías son complementarias: hashlib verifica integridad, yara-python detecta malware e impacket analiza protocolos de red. Juntas conforman una cadena de análisis defensivo coherente.

7. Parte Práctica: Análisis Forense de APKs

7.1. Plataforma Utilizada

Se utilizó un entorno Linux con Python 3.14, un entorno virtual (venv) con yara-python y androguard, más las herramientas apktool, msfvenom y Metasploit Framework para generar el APK envenenado. Los notebooks de Google Colab y el repositorio GitHub complementan la práctica.

7.2. Descripción del Problema

Determinar si un APK ha sido manipulado con payload malicioso inyectado por MsfVenom. Se analizan tres archivos: `CairosOriginal.apk` (Flutter legítima, 52.4 MB), `CairosVenom.apk` (misma app con payload meterpreter_reverse_tcp) y `payload.apk` (APK standalone de msfvenom, 10 KB). El análisis consta de dos fases: (1) comparación de hashes criptográficos, y (2) detección automatizada con YARA + Androguard.

7.3. Datos Utilizados

- `CairosOriginal.apk`: 52,400,958 bytes, 2 permisos.
- `CairosVenom.apk`: 52,431,637 bytes, 24 permisos, subpaquete `dfwto`.
- `payload.apk`: 10,246 bytes, 23 permisos.

7.4. Código Desarrollado: Fase 1 (hashlib)

El script `comparar_hashes.py` implementa la comparación criptográfica de los APK. Su estructura se detalla en los Listados 3 y 4 de la Sección 4.1.6.

7.5. Código Desarrollado: Fase 2 (YARA + Androguard)

El script `detector_yara.py` implementa un motor de detección multicapa. Su estructura de reglas se detalla en el Listado 6 y el análisis heurístico en el Listado 7 de la Sección 4.2.6. El sistema asigna una puntuación de riesgo que combina todos los indicadores detectados.

7.6. Explicación del Código

La Fase 1 utiliza lectura por bloques de 4 KB para procesar archivos de más de 50 MB sin saturar la RAM, calculando cuatro algoritmos en una sola pasada (0.37 segundos). La comparación XOR bit a bit del SHA-256 cuantifica el efecto avalancha.

La Fase 2 emplea detección por capas: (a) reglas YARA que buscan firmas de RAT en el DEX (reflexión, red, servicios); (b) análisis del `AndroidManifest.xml` con Androguard para contar

permisos peligrosos; (c) detección de subpaquetes con nombres aleatorios de 5 caracteres generados por MsfVenom; (d) comparación del tamaño del DEX. Cada indicador suma puntos; si el total supera 5, el APK se clasifica como ENVENENADO (Huacac Soto, 2026).

7.7. Resultados Obtenidos

Los resultados completos se presentan en la Tabla 8 de la Sección 4.2.7. En resumen: CairoOriginal.apk fue clasificado LIMPIO (2 permisos, sin subpaquetes); CairoVenom.apk fue clasificado ENVENENADO (24 permisos, 14 peligrosos, subpaquete dfwt0); payload.apk fue clasificado ENVENENADO (23 permisos, 14 peligrosos).

Adicionalmente, los hashes criptográficos de la Tabla 5 confirmaron que todos los algoritmos producen valores diferentes entre el original y el envenenado, validando la detección de modificaciones.

7.8. Enlaces y Evidencias

- Repositorio GitHub: <https://github.com/AlvaroHuacacSoto/hashlib-Jara>
- Notebook hashlib: <https://colab.research.google.com/drive/1oXyDNrLdfR4-8o1tR1WURTp8>
- Notebook yara-python: https://colab.research.google.com/drive/173T7NiHw52sU_ySdK0RQoJzm5KdpLAmp

8. Caso Práctico: Detección de APK Envenenado

8.1. Descripción del Caso

Un desarrollador independiente descubre que una versión modificada de su aplicación CairoOriginal.apk circula en foros de terceros. La versión legítima es una app Flutter de 52 MB con solo 2 permisos. La versión sospechosa solicita 24 permisos, incluyendo acceso a SMS, cámara, micrófono y contactos. El desarrollador necesita determinar técnicamente si el APK fue manipulado.

8.2. Problema Identificado

- El APK circulante solicita 24 permisos (14 peligrosos) frente a 2 del original.
- Los hashes criptográficos no coinciden con la versión legítima.
- Se sospecha inyección de RAT mediante Metasploit/MsfVenom.
- Se necesita un método automatizado y reproducible de detección.

8.3. Solución Propuesta

Se desarrollaron dos scripts Python complementarios:

- **comparar_hashes.py**: calcula MD5, SHA-1, SHA-256 y SHA-512 de ambos APK. La diferencia en los cuatro algoritmos confirma alteración. El XOR del SHA-256 revela 50.8 % de bits cambiados.
- **detector_yara.py**: motor de detección que combina reglas YARA sobre el DEX con heurísticas del manifiesto (permisos, subpaquetes). Emite veredicto LIMPIO/ENVENENADO con puntuación.

8.4. Beneficios de la Solución

- Verificación de integridad en segundos mediante comparación de hash.
- Detección automatizada sin ingeniería inversa manual.
- Reproducible: clonar repositorio, instalar dependencias, ejecutar.
- Aplicable a detección de malware en tiendas alternativas y campañas de phishing.

Tabla 13: Resumen del caso práctico

Elemento	Descripción
Contexto	Desarrollador descubre APK alterado en distribución no autorizada.
Problema	24 permisos vs 2 originales; hashes no coinciden; payload Metasploit.
Archivos	CairosOriginal.apk, CairosVenom.apk, payload.apk.
Librerías	hashlib (integridad), yara-python + androguard (detección), impacket (documentación teórica).
Herramienta de ataque	MsfVenom con payload andro-id/meterpreter/reverse_tcp.
Resultado	Original LIMPIO, envenenado ENVENENADO con 14 indicadores.

9. Conclusiones

1. Las librerías hashlib, yara-python e impacket demuestran que Python es una plataforma robusta para ciberseguridad defensiva. Cada librería aborda una dimensión diferente: integridad (hashlib), detección (yara-python) y análisis de protocolos (impacket). El proyecto práctico con APKs envenenados validó las dos primeras en un escenario real; impacket fue documentada teóricamente por las barreras de autenticación que requiere.
2. El análisis de hashes confirmó el efecto avalancha: los cuatro algoritmos produjeron valores diferentes entre el APK original y el envenenado, con 130 de 256 bits (50.8 %) distintos en SHA-256. La lectura por bloques de 4 KB procesó archivos de 52 MB en 0.37 segundos calculando cuatro hashes simultáneos.

3. El detector clasificó correctamente los tres APK: original como LIMPIO (2 permisos), envenenado como ENVENENADO (24 permisos, subpaquete inyectado dfwto) y payload standalone como ENVENENADO. La combinación de reglas YARA con heurísticas de Androguard fue más efectiva que cualquiera de las técnicas por separado.
4. Herramientas de código abierto como MsfVenom pueden generar APKs maliciosos indetectables a simple vista, pero las librerías Python permiten identificarlos mediante análisis automatizado de su estructura interna. Esto demuestra el valor de la ciberseguridad defensiva basada en software libre.

10. Recomendaciones

- Clonar el repositorio <https://github.com/AlvaroHuacacSoto/hashlib-Jara> que contiene todos los scripts, APK de prueba y este documento.
- Revisar los notebooks de Google Colab como complemento interactivo.
- Usar SHA-256 o SHA-512 sobre MD5/SHA-1 en aplicaciones de seguridad, por sus vulnerabilidades de colisión conocidas.
- Combinar reglas YARA con análisis estructural del manifiesto Android para reducir falsos negativos en detección de malware móvil.
- Para impacket, montar un laboratorio controlado con Windows Server y obtener autorización formal antes de ejecutar cualquier script.
- Mantener actualizadas las reglas YARA con firmas de repositorios comunitarios como YARA-Rules y VirusTotal.
- Aplicar principios éticos y legales en toda práctica: obtener autorización escrita antes de auditar sistemas externos.

Referencias

- [1] Álvarez, V. M. (2014). *YARA: The pattern matching swiss knife for malware researchers*. VirusTotal Blog. <https://www.virustotal.com/gui/blog>
- [2] Androguard. (2024). *Androguard: Reverse engineering tool for Android applications* (Versión 4.1) [Software]. GitHub. <https://github.com/androguard/androguard>
- [3] Fortra. (2024). *impacket: A collection of Python classes for working with network protocols* [Software]. GitHub. <https://github.com/fortra/impacket>
- [4] Huacac Soto, A. (2026). *hashlib-Jara: Detección de APKs envenenados con Python* [Repositorio de código fuente]. GitHub. <https://github.com/AlvaroHuacacSoto/hashLib-Jara>

- [5] Ley N.º 30096. (2013). *Ley de Delitos Informáticos*. Congreso de la República del Perú. <https://www.gob.pe/institucion/congreso-de-la-republica/normas-legales>
- [6] Python Software Foundation. (2024). *hashlib — Secure hashes and message digests*. Python 3.12 Documentation. <https://docs.python.org/3/library/hashlib.html>
- [7] Rapid7. (2024). *Metasploit Framework Documentation*. <https://docs.metasploit.com/>
- [8] SecureAuth Corporation. (2023). *Impacket scripts and examples for network protocol analysis* [Software]. GitHub. <https://github.com/fortra/impacket/tree/master/examples>
- [9] Van Rossum, G. y Drake, F. L. (2009). *Python 3 Reference Manual*. Python Software Foundation. <https://docs.python.org/3/reference/>
- [10] VirusTotal / Google. (2024). *YARA documentation* (Versión 4.5). <https://virustotal.github.io/yara/>

Anexos

Anexo A: Código Fuente Completo

El código fuente completo de `comparar_hashes.py` y `detector_yara.py` está disponible en el repositorio: <https://github.com/AlvaroHuacacSoto/hashlib-Jara>

Anexo B: Reglas YARA Utilizadas

Listing 9: Regla YARA: dex_metasploit_payload

```
1 rule dex_metasploit_payload {
2     meta:
3         description = "Detecta payload de Metasploit en DEX"
4         author = "Lab Seguridad"
5         severity = "critical"
6     strings:
7         $a = "Ljava/lang/reflect/Method;"
8         $b = "Ljava/net/URLConnection;"
9         $c = "addRequestProperty"
10        $d = ";>startService(Landroid/content/Context;)V"
11        $e = ";>openConnection()Ljava/net/URLConnection;"
12    condition:
13        uint32(0) == 0x0A786564 and
```

```
14     4 of them
15 }
```

Listing 10: Regla YARA: dex_payload_backdoor

```
1 rule dex_payload_backdoor {
2     meta:
3         description = "Estructura tipica de RAT/backdoor Android"
4         severity = "high"
5     strings:
6         $ref = "Ljava/lang/reflect/Method;"
7         $start = ";>startService(Landroid/content/Context;)V"
8     condition:
9         uint32(0) == 0x0A786564 and
10        all of them
11 }
```

Anexo C: Enlaces a Google Colab

- hashlib: <https://colab.research.google.com/drive/1oXyDNrLdfr4-8oltR1WURTp8IMK65TTF>
- yara-python: https://colab.research.google.com/drive/173T7NiHw52sU_ySdk0RQoJzm5Kdp